

C++ Mock Interview — 2 Ժամանոց Մեծ Թեստ

Ամբողջական հարցազրույցի սիմուլյացիա (Junior / Mid C++)

Այս թեստը իրական 2-ժամանոց տեխնիկական հարցազրույցի սիմուլյացիա է: Ընդգրկում է բոլոր հիմնական թեմաները՝ C++ հիմունքներ, OOP, smart pointers, memory, concurrency, exceptions, STL, data structures, algorithms, Linux:

Ընդհանուր՝ **120 հարց, 120 րոպե**: Բաժանված է 6 բլոկի՝ ամեն բլոկը ~20 րոպե:

Կանոններ. - Ամեն հարցի վրա միջինում 1 րոպե: - Պատասխանները **միայն վերջի էջերում** են (Answer Keys): - Սկզբում ինքդ լուծիր, հետո ստուգիր: - Ամեն բլոկի վերջում գրիր քո ժամանակը ու ճիշտ պատասխանների քանակը:

ԲԼՈԿ 1 (0-20 րոպե):	C++ Basics & Memory	(Q1-Q20)
ԲԼՈԿ 2 (20-40 րոպե):	OOP & Virtual	(Q21-Q40)
ԲԼՈԿ 3 (40-60 րոպե):	Smart Pointers & RAII	(Q41-Q60)
ԲԼՈԿ 4 (60-80 րոպե):	STL & Data Structures	(Q61-Q80)
ԲԼՈԿ 5 (80-100 րոպե):	Concurrency & Exceptions	(Q81-Q100)
ԲԼՈԿ 6 (100-120 րոպե):	Algorithms, Linux, Mixed	(Q101-Q120)

ԲԼՈԿ 1 — C++ Basics & Memory (Q1-Q20)

Q1

Ի՞նչ է constructor-ը: - (a) object ջնջելիս կանչվող - (b) object ստեղծելիս ավտոմատ կանչվող - (c) static function - (d) operator

Q2

Constructor-ը return type ունի: - (a) void - (b) int - (c) չունի - (d) class-ի type

Q3

Destructor-ը կարող է parameter ունենալ: - (a) այո - (b) ոչ - (c) միայն int - (d) միայն default

Q4

```
int* a = new int;  
cout << *a;
```

- (a) 0
- (b) undefined (աղբ)
- (c) CE
- (d) nullptr

Q5

```
int* b = new int();  
cout << *b;
```

- (a) undefined
- (b) 0
- (c) CE
- (d) 1

Q6

new[]-ի գույգը: - (a) delete - (b) delete[] - (c) free - (d) free[]

Q7

`std::move` -ը իրականում ինչ է անում: - (a) տեղափոխում տվյալները - (b) cast է rvalue-ի (static_cast) - (c) ջնջում - (d) copy

Q8

```
int&& r = 5;
```

`r` -ը հետո օգտագործելիս ինչ է: - (a) rvalue - (b) lvalue (անուն ունի) - (c) pointer - (d) const

Q9

```
void f(std::string& s){}  
f("hello");
```

- (a) OK

- (b) compile error (non-const ref temporary չի ընդունում)
- (c) runtime error
- (d) տպում hello

Q10

`const int* p` — ինչն է const: - (a) pointer-ը - (b) տվյալը - (c) երկուսն էլ - (d) ոչ մեկը

Q11

`int* const p` — ինչն է const: - (a) տվյալը - (b) pointer-ը - (c) երկուսն էլ - (d) ոչ մեկը

Q12

Static local variable-ը երբ է ստեղծվում: - (a) ամեն կանչ - (b) մեկ անգամ (առաջին կանչ) - (c) երբեք - (d) program start

Q13

`const` vs `constexpr` : - (a) նույնը - (b) `constexpr` = compile-time, `const` = immutable - (c) `const` = compile-time - (d) `constexpr` = runtime

Q14

Memory leak ի՞նչ է: - (a) շատ memory - (b) heap memory վերցված, չի ազատվել, հասցեն կորել - (c) stack overflow - (d) null pointer

Q15

Dangling pointer ի՞նչ է: - (a) null pointer - (b) ջնջված memory-ի ցույց տվող pointer - (c) մեծ pointer - (d) shared

Q16

Double delete ի՞նչ է: - (a) 2 new - (b) նույն pointer 2 անգամ delete (UB) - (c) delete[] - (d) free

Q17

new vs malloc — կարևոր տարբերություն: - (a) ոչ մի - (b) new-ը ctor/dtor կանչում, malloc-ը ոչ - (c) malloc արագ - (d) new-ը C-ի

Q18

new անհաջողություն -> ? - (a) nullptr - (b) throw std::bad_alloc - (c) crash - (d) 0

Q19

`std::string_view`-ը ինչ է պահում: - (a) string-ի copy - (b) pointer + length - (c) միայն length - (d) heap

Q20

SSO (Small String Optimization) ինչ է անում: - (a) փոքր string-ը object-ի ներսում, heap չի գնում - (b) compress - (c) միշտ heap - (d) ջնջում

ՔԼՈԿ 2 — OOP & Virtual (Q21-Q40)

Q21

OOP-ի 4 սյունները: - (a) Class, Object, Method, Field - (b) Encapsulation, Inheritance, Polymorphism, Abstraction - (c) public, private, protected, default - (d) compile, runtime, static, dynamic

Q22

Encapsulation-ը հիմնականում: - (a) reuse code - (b) hide data - (c) hide implementation - (d) many forms

Q23

Runtime polymorphism C++-ում ինչով: - (a) function overloading - (b) operator overloading - (c) virtual functions - (d) templates

Q24

```
class A { public: void f(){cout<<"A";} };
class B : public A { public: void f(){cout<<"B";} };
A* p = new B(); p->f();
```

- (a) A (static binding, virtual չկա)
- (b) B
- (c) CE
- (d) AB

Q25

Նույնը, բայց `virtual void f()` . Ի՞նչ կտալի: - (a) A - (b) B - (c) CE - (d) ոչինչ

Q26

Pure virtual function syntax: - (a) virtual void f(); - (b) virtual void f() = 0; - (c) void f() = 0; - (d) abstract void f();

Q27

Abstract class-ից object: - (a) կարելի - (b) չի կարելի (compile error) - (c) միայն pointer - (d) միայն static

Q28

Ինչո՞ւ virtual destructor: - (a) արագ - (b) base pointer-ով delete -> derived destructor էլ կանչվի - (c) memory խնայում - (d) պարտադիր ամեն class

Q29

vtable-ը ում է պատկանում: - (a) object-ին - (b) class-ին (մեկ per polymorphic class) - (c) function-ին - (d) vptr-ին

Q30

vptr-ը որտեղ է պահվում: - (a) class-ում - (b) object-ի ներսում - (c) stack-ում - (d) vtable-ում

Q31

Overloading vs Overriding — overloading-ը երբ է որոշվում: - (a) runtime - (b) compile-time - (c) link-time - (d) երբեք

Q32

```
class Base { virtual void f(int){} };  
class Derived : public Base { void f(double){} };
```

Derived-ի f(double)-ը Base-ի նկատմամբ: - (a) override - (b) overload - (c) ոչ override ոչ overload (name hiding) - (d) CE

Q33

Constructor-ի հերթականությունը inheritance-ում: - (a) Derived -> Base - (b) Base -> Derived - (c) միաժամանակ - (d) պատահական

Q34

Destructor-ի հերթականությունը: - (a) Base -> Derived - (b) Derived -> Base - (c) միաժամանակ - (d) պատահական

Q35

"Car has an Engine" կապը: - (a) is-a (inheritance) - (b) has-a (composition) - (c) uses-a - (d) static

Q36

Diamond Problem-ը C++-ում ինչով է լուծվում: - (a) interface - (b) virtual inheritance - (c) friend - (d) static

Q37

Constructor-ը կարող է virtual լինել: - (a) այո - (b) ոչ (object դեռ գոյություն չունի) - (c) միայն abstract - (d) միայն C++11

Q38

<< operator-ը սովորաբար: - (a) member - (b) non-member (friend) - (c) static - (d) virtual

Q39

Postfix `operator++(int)` -ի `int` -ը ինչի համար է: - (a) արժեք ավելացնել - (b) prefix-ից տարբերել (dummy) - (c) index - (d) ոչնչի

Q40

SOLID-ի "S"-ը: - (a) Static - (b) Single Responsibility - (c) Simple - (d) Standard

ՔԼՈԿ 3 — Smart Pointers & RAII (Q41-Q60)

Q41

RAII ինչ է նշանակում: - (a) Random Access... - (b) Resource Acquisition Is Initialization - (c) Rapid Allocation... - (d) Reference And Index Init

Q42

RAII-ում resource-ը երբ է ազատվում: - (a) constructor-ում - (b) destructor-ում (scope-ի վերջ) - (c) main-ի վերջ - (d) ձեռքով

Q43

unique_ptr-ի ownership: - (a) shared - (b) exclusive (մեկ owner) - (c) non-owning - (d) weak

Q44

unique_ptr copy լինում է: - (a) այո - (b) ոչ (միայն move) - (c) միայն int - (d) միշտ

Q45

unique_ptr ownership ինչպես ես փոխանցում: - (a) copy - (b) std::move - (c) get() - (d) reset()

Q46

shared_ptr-ը երբ է delete անում resource-ը: - (a) առաջին owner ոչնչանալիս - (b) վերջին owner (count 0) - (c) երբեք - (d) main վերջ

Q47

use_count() ինչ է տալիս: - (a) memory չափ - (b) քանի strong owner - (c) քանի weak - (d) bucket

Q48

make_shared-ի առավելությունը: - (a) 2 allocation - (b) 1 allocation (resource+control block) - (c) դանդաղ - (d) ոչ մի

Q49

shared_ptr-ի ամենամեծ խնդիր: - (a) արագ - (b) circular reference (leak) - (c) copy չկա - (d) null

Q50

Circular reference-ի լուծում: - (a) unique_ptr - (b) weak_ptr - (c) raw pointer - (d) malloc

Q51

weak_ptr ավելացնում է strong count-ը: - (a) այո - (b) ոչ - (c) միայն copy - (d) միշտ

Q52

weak_ptr-ից resource-ի անվտանգ մուտք: - (a) *wp - (b) wp.lock() - (c) wp.get() - (d) wp->

Q53

expired() ինչ է տալիս: - (a) memory չափ - (b) resource ջնջված է? (true/false) - (c) count - (d) bucket

Q54

Default smart pointer ընտրություն: - (a) shared_ptr - (b) unique_ptr - (c) weak_ptr - (d) raw

Q55

Rule of Zero ինչ է: - (a) 0 member - (b) RAII տիպեր -> ոչ մի ձեռքով dtor/copy/move - (c) 0 constructor - (d) 0 allocation

Q56

Rule of Three-ի երեքը: - (a) 3 ctor - (b) dtor, copy-ctor, copy-assign - (c) 3 pointer - (d) 3 member

Q57

shared_ptr reference count-ը ինչ է: - (a) սովորական int - (b) atomic (thread-safe) - (c) float - (d) չկա

Q58

lock_guard-ի առավելությունը: - (a) արագ - (b) RAII - lock ctor/unlock dtor - (c) քիչ memory - (d) atomic

Q59

```
auto a = std::make_unique<int>(5);  
auto b = a;
```

- (a) OK
- (b) compile error (unique copy չկա)
- (c) runtime error
- (d) leak

Q60

```
auto p = std::make_shared<int>(10);  
auto q = p;  
cout << p.use_count();
```

- (a) 1
- (b) 2
- (c) 0
- (d) CE

ԲՆԿ 4 — STL & Data Structures (Q61-Q80)

Q61

vector-ը ինչ է ներսում: - (a) linked list - (b) dynamic array (contiguous) - (c) tree - (d) hash

Q62

list-ը ինչ է C++-ում: - (a) dynamic array - (b) doubly linked list - (c) tree - (d) hash

Q63

vector-ի random access: - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q64

list-ի random access (index): - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) չկա

Q65

size vs capacity: - (a) նույնը - (b) size = element-ներ, capacity = allocated memory - (c) size = memory - (d) capacity = element-ներ

Q66

reserve() vs resize(): - (a) նույնը - (b) reserve = capacity, resize = element-ներ ստեղծում - (c) reserve element ստեղծում - (d) resize = capacity

Q67

map-ը ինչ tree է: - (a) AVL - (b) Red-Black Tree - (c) B-Tree - (d) Trie

Q68

map insert complexity: - (a) $O(1)$ - (b) $O(\log n)$ - (c) $O(n)$ - (d) $O(n \log n)$

Q69

unordered_map insert (average): - (a) $O(\log n)$ - (b) $O(1)$ - (c) $O(n)$ - (d) $O(n^2)$

Q70

lower_bound() նր container-ում կա: - (a) unordered_map - (b) map/set (sorted) - (c) vector always - (d) stack

Q71

find() vs operator[] map-ում — operator[]: - (a) չի ստեղծում - (b) ստեղծում է key եթե չկա - (c) throw - (d) CE

Q72

Stack-ը ինչ սկզբունքով: - (a) FIFO - (b) LIFO - (c) sorted - (d) random

Q73

Queue-ն ինչ սկզբունքով: - (a) LIFO - (b) FIFO - (c) sorted - (d) random

Q74

BFS-ի համար նր DS: - (a) stack - (b) queue - (c) heap - (d) trie

Q75

BST inorder traversal-ը տալիս է: - (a) random - (b) sorted - (c) reverse - (d) level

Q76

Max-heap-ում root-ը: - (a) ամենափոքր - (b) ամենամեծ - (c) միջին - (d) random

Q77

priority_queue default-ով: - (a) min-heap - (b) max-heap - (c) balanced - (d) sorted

Q78

push_back complexity: - (a) $O(n)$ always - (b) amortized $O(1)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q79

Vector iterator invalidation՝ push_back-ից հետո: - (a) երբեք - (b) reallocation-ի դեպքում invalid - (c) միշտ valid - (d) CE

Q80

erase-remove idiom: - (a) `v.erase(x)` - (b) `v.erase(remove(begin,end,x), end())` - (c) `v.remove(x)` - (d) `v.clear()`

ԲԼՈՎ 5 — Concurrency & Exceptions (Q81-Q100)

Q81

Thread ինչ է: - (a) առանձին ծրագիր - (b) ծրագրի ամենափոքր կատարողական միավոր - (c) memory - (d) file

Q82

Process vs thread — memory: - (a) process shared - (b) process առանձին, thread shared - (c) երկուսն shared - (d) երկուսն առանձին

Q83

join() vs detach() — join: - (a) thread ջնջում - (b) սպասում մինչև thread ավարտվի - (c) նոր thread - (d) detach

Q84

Mutex ինչ է: - (a) memory - (b) Mutual Exclusion - մեկ thread միանգամից - (c) thread - (d) pointer

Q85

Race condition ինչ է: - (a) thread արագ - (b) մի քանի thread նույն data փոխում, արդյունք կարգից կախված - (c) deadlock - (d) leak

Q86

counter++ ինչու race: - (a) atomic է - (b) 3 քայլ (read/+1/write) - (c) դանդաղ - (d) չկա race

Q87

Deadlock ինչ է: - (a) crash - (b) երկու+ thread սպասում իրար անվերջ - (c) leak - (d) race

Q88

Deadlock-ի ամենապարզ լուծում: - (a) ավելի mutex - (b) նույն կարգով lock - (c) detach - (d) atomic

Q89

atomic vs mutex — երբ atomic: - (a) բարդ logic - (b) փոքր operation (counter/flag) - (c) container - (d) միշտ mutex

Q90

lock_guard-ի vs unique_lock: - (a) նույնը - (b) unique_lock ճկուն (ձեռքով lock/unlock) - (c) lock_guard ճկուն - (d) unique_lock հին

Q91

Exception ինչ է: - (a) memory leak - (b) error handling (throw/try/catch) - (c) thread - (d) pointer

Q92

Stack unwinding-ի ժամանակ ինչ է կանչվում: - (a) constructor - (b) local object-ների destructor-ներ - (c) main - (d) ոչինչ

Q93

catch by value-ի խնդիր: - (a) արագ - (b) slicing + copy - (c) leak - (d) ոչ մի

Q94

Destructor-ից throw անել: - (a) OK - (b) վտանգավոր (std::terminate) - (c) արագ - (d) leak

Q95

noexcept function throw անելիս: - (a) OK - (b) std::terminate - (c) catch - (d) return

Q96

vector::at() out of range -> ? - (a) UB - (b) throw out_of_range - (c) nullptr - (d) 0

Q97

vector operator[] out of range -> ? - (a) throw - (b) undefined behavior - (c) 0 - (d) nullptr

Q98

std::exception-ի հիմնական մեթոդ: - (a) message() - (b) what() - (c) error() - (d) msg()

Q99

```
try { throw 42; }  
catch (const std::exception& e) {}
```

- (a) caught
- (b) terminate (int չի ժառանգում exception)
- (c) 42
- (d) OK

Q100

Custom exception ինչպես: - (a) throw int - (b) ժառանգել std::exception, override what() - (c) throw string - (d) չկարելի

ԲԼՈԿ 6 — Algorithms, Linux, Mixed (Q101-Q120)

Q101

Big O — ամենաարագ: - (a) $O(n)$ - (b) $O(1)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q102

Binary search complexity: - (a) $O(n)$ - (b) $O(\log n)$ - (c) $O(1)$ - (d) $O(n \log n)$

Q103

Binary search-ի պայմանը: - (a) unsorted - (b) sorted array - (c) linked list - (d) hash

Q104

Efficient sort complexity (merge/quick): - (a) $O(n)$ - (b) $O(n \log n)$ - (c) $O(n^2)$ - (d) $O(\log n)$

Q105

Quick sort worst case: - (a) $O(n \log n)$ - (b) $O(n^2)$ - (c) $O(n)$ - (d) $O(\log n)$

Q106

Merge sort-ը stable է: - (a) այո - (b) ոչ - (c) երբեմն - (d) CE

Q107

Հաշված sort $O(n \log n)$ -ից արագ (non-comparison): - (a) bubble - (b) counting/radix - (c) quick - (d) heap

Q108

`ls -l` ինչ է տալիս: - (a) hidden ֆայլեր - (b) մանրամասն (permissions, size, date) - (c) recursive - (d) sorted

Q109

`cd ..` ինչ է: - (a) home - (b) մեկ վերև (parent) - (c) նախորդ - (d) root

Q110

`grep -r` ինչ է: - (a) case-insensitive - (b) recursive search - (c) invert - (d) count

Q111

`grep -v` ինչ է: - (a) version - (b) հակառակ (առանց pattern) - (c) verbose - (d) count

Q112

`find . -name "*.txt"` ինչ է անում: - (a) ջնջում .txt - (b) գտնում բոլոր .txt recursive - (c) տպում - (d) copy

Q113

Process ինչ է: - (a) thread - (b) աշխատող ծրագիր, առանձին memory, PID - (c) file - (d) memory block

Q114

`kill -9` ինչ է: - (a) pause - (b) SIGKILL (բռնով սպանել) - (c) restart - (d) resume

Q115

Two Sum խնդրի օպտիմալ լուծում: - (a) $O(n^2)$ brute force - (b) $O(n)$ hashmap - (c) $O(n \log n)$ - (d) $O(1)$

Q116

Linked list-ի cycle detection: - (a) hashmap only - (b) Floyd's (fast/slow pointer) - (c) sort - (d) binary search

Q117

Reverse linked list-ի complexity: - (a) $O(1)$ - (b) $O(n)$ - (c) $O(\log n)$ - (d) $O(n^2)$

Q118

Valid parentheses-ի համար DS: - (a) queue - (b) stack - (c) heap - (d) map

Q119

Adjacency list vs matrix — sparse graph-ի համար: - (a) matrix - (b) list ($O(V+E)$) - (c) նույնը - (d) heap

Q120

Dijkstra's shortest path DS: - (a) stack - (b) min-heap (priority_queue) - (c) trie - (d) hash

ANSWER KEY — PLY 1 (C++ Basics & Memory)

1-b · 2-c · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-b · 12-b · 13-b · 14-b · 15-b · 16-b · 17-b · 18-b · 19-b · 20-a

ANSWER KEY — PLY 2 (OOP & Virtual)

21-b · 22-b · 23-c · 24-a · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b · 31-b · 32-c · 33-b · 34-b · 35-b · 36-b · 37-b · 38-b · 39-b · 40-b

ANSWER KEY — PLY 3 (Smart Pointers & RAI)

41-b · 42-b · 43-b · 44-b · 45-b · 46-b · 47-b · 48-b · 49-b · 50-b · 51-b · 52-b · 53-b · 54-b · 55-b · 56-b · 57-b · 58-b · 59-b · 60-b

ANSWER KEY — PLY 4 (STL & Data Structures)

61-b · 62-b · 63-a · 64-b · 65-b · 66-b · 67-b · 68-b · 69-b · 70-b · 71-b · 72-b · 73-b · 74-b · 75-b · 76-b · 77-b · 78-b · 79-b · 80-b

ANSWER KEY — PLY 5 (Concurrency & Exceptions)

81-b · 82-b · 83-b · 84-b · 85-b · 86-b · 87-b · 88-b · 89-b · 90-b · 91-b · 92-b · 93-b · 94-b · 95-b · 96-b · 97-b · 98-b · 99-b · 100-b

ANSWER KEY — ՔԼՈԿ 6 (Algorithms, Linux, Mixed)

101-b · 102-b · 103-b · 104-b · 105-b · 106-a · 107-b · 108-b · 109-b · 110-b 111-b · 112-b · 113-b · 114-b · 115-b · 116-b · 117-b · 118-b · 119-b · 120-b

Գնահատում (Scoring)

Ճիշտ պատասխաններ:

108-120 (90%+)

-> Գերազանց. պատրաստ ես mid-level հարցազրույցի

96-107 (80%+)

-> Շատ լավ. junior հարցազրույցի մեծ մասը կփակես

84-95 (70%+)

-> Լավ. կրկնիր թույլ բլոկերը

72-83 (60%+)

-> Բավարար. պետք է ավելի շատ պատրաստվել

<72 (<60%)

-> Կրկնիր հիմունքները (առանձին guide-երը)

Բլոկ առ բլոկ վերլուծություն

ՔԼՈԿ 1 (Basics/Memory):

___ / 20

Ժամանակ: ___ րոպե

ՔԼՈԿ 2 (OOP/Virtual):

___ / 20

Ժամանակ: ___ րոպե

ՔԼՈԿ 3 (Smart Ptr/RAII):

___ / 20

Ժամանակ: ___ րոպե

ՔԼՈԿ 4 (STL/DS):

___ / 20

Ժամանակ: ___ րոպե

ՔԼՈԿ 5 (Concurrency/Exc):

___ / 20

Ժամանակ: ___ րոպե

ՔԼՈԿ 6 (Algo/Linux):

___ / 20

Ժամանակ: ___ րոպե

ԸՆԴԱՄԵՆԸ:

___ / 120

Ամեն թույլ բլոկի համար կրկնիր համապատասխան առանձին presentation-ը (Constructor, OOP, Smart Pointers, STL, Concurrency, Exceptions, Algorithms, Linux):

Հաջողություն հարցազրույցին: